PROJET FINAL DE FIN DE MODULE

Application de Gestion d'un Centre de Formation

C# Oriente Objet + WinForms

Remise du projet	Livrables attendus
Email : aboubacarsawadogo15@gmail.com **Objet : Projet dotnet c# 2026** Format : .zip ou .rar compresse	- Dossier Visual Studio complet - Fichiers de donnees JSON ou texte - Captures d'ecran de l'interface - Fichier README.md

BAREME --- 100 POINTS	
Structure Orientee Objet (classes, heritage, constructeurs, proprietes)	**25 pts**
Encapsulation et logique metier (validations, methodes utiles)	**15 pts**
Interface graphique WinForms (formulaires fonctionnels, affichage)	**20 pts**
Fonctionnalites globales (CRUD, notes, statistiques)	**20 pts**
Sauvegarde des donnees (lecture/ecriture fichiers)	**10 pts**
Presentation orale (demonstration, clarte, maitrise du code)	**10 pts**

1. CONTEXTE ET OBJECTIFS

Un centre de formation souhaite une application pour gerer ses etudiants, formateurs, modules, notes et inscriptions. Vous devez concevoir et developper cette application en C# oriente objet avec une interface graphique WinForms.

Ce projet doit vous permettre de demontrer votre maitrise de :

- La conception en classes et objets
- L'heritage entre classes
- L'encapsulation avec validations
- Les membres static
- La separation logique metier / interface graphique
- La sauvegarde de donnees dans des fichiers

IMPORTANT :

Ce projet doit etre votre propre travail. Vous devez etre capables d'expliquer chaque ligne de code que vous ecrivez le jour de la presentation.
Tout code incompris ou copie sans maitrise sera detecte lors de la demonstration.

2. MODELISATION DES CLASSES

Avant de coder, modelisez votre application. Chaque classe doit etre dans son propre fichier .cs. Voici ce que chaque classe doit contenir au minimum.

2.1 Classe Personne — classe mere

Role : classe de base dont heritent Etudiant et Formateur.

Attributs minimum : Id, Nom, Prenom, Age
Constructeur : initialiser tous les attributs
Proprietes : utiliser des get/set pour chaque attribut
Validation : l'age NE PEUT PAS etre negatif (a gerer dans le set)
Membre static : un compteur du nombre total de personnes creees
Methode : une methode AfficherDetails() qui retourne une description de la personne

2.2 Classe Etudiant — heritee de Personne

Role : represente un etudiant avec ses notes.

Attributs supplementaires : Matricule, Niveau
Collection : une liste de notes (List<Note> ou similaire)
Constructeur : appeler le constructeur parent avec le mot-cle base(...)
Methode : AjouterNote(Note note) pour ajouter une note a la liste
Methode : CalculerMoyenne() qui calcule la moyenne ponderee par les coefficients
Override : redefinir AfficherDetails() pour inclure matricule, niveau et moyenne

2.3 Classe Formateur — heritee de Personne

Role : represente un formateur du centre.

Attributs supplementaires : Specialite, ModuleEnseigne
Attribut : Salaire (mensuel)
Constructeur : appeler le constructeur parent avec base(...)
Validation : le salaire DOIT etre strictement positif (dans le set)
Methode : CalculerSalaireAnnuel() qui retourne le salaire annuel

2.4 Classe Module

Role : represente un module de formation.

Attributs : Code, Libelle, Duree (en heures), Coefficient
Validation : la duree DOIT etre strictement superieure a 0 (dans le set)
Membre static : un compteur du nombre total de modules crees

Constructeur : initialiser tous les attributs et incrementer le compteur

2.5 Classe Note

Role : represente la note d'un etudiant pour un module.

Attributs : reference vers l'Etudiant concerne, reference vers le Module concerne
Attribut : Valeur (la note entre 0 et 20)
Validation : la valeur DOIT etre entre 0 et 20 inclus (dans le set)
Constructeur : initialiser etudiant, module et valeur

2.6 Classe GestionCentre

Role : centralise toutes les donnees et les operations metier.

Listes : contient les listes d'etudiants, formateurs, modules, notes
Chargement : le constructeur charge les donnees depuis les fichiers
Methodes : AjouterEtudiant(), AjouterFormateur(), AjouterModule()
Methodes : SupprimerEtudiant(), ModifierEtudiant()
Methodes : MeilleurEtudiant(), MoyenneGenerale()
Principe : cette classe ne doit JAMAIS contenir de code WinForms

3. HERITAGE ET ENCAPSULATION

3.1 Comment fonctionne l'heritage en C#

L'heritage permet a une classe fille d'heriter des attributs et methodes de sa classe mere. En C#, on utilise le signe deux-points (:) pour declarer l'heritage.

Syntaxe de l'heritage :

```
public class ClasseFille : ClasseMere { ... }

Pour appeler le constructeur de la classe mere depuis la classe fille :
public ClasseFille(parametres) : base(parametresParent) { ... }

Pour redefinir une methode heritee :
public override string MaMethode() { ... }

Pour appeler la methode du parent dans le override :
return base.MaMethode() + " informations supplementaires";
```

Application dans votre projet :

- Personne est la classe mere (mot-cle virtual sur les methodes a redefinir)
- Etudiant herite de Personne -> class Etudiant : Personne
- Formateur herite de Personne -> class Formateur : Personne
- Chaque classe fille appelle base(...) dans son constructeur
- AfficherDetails() est definie dans Personne et surchargee dans les classes filles

3.2 Comment fonctionne l'encapsulation

L'encapsulation consiste a proteger les donnees en controlant leur acces via des proprietes. Une propriete avec get/set permet de valider les valeurs avant de les stocker.

Structure d'une propriete avec validation :

```
private int _monChamp;  // champ prive

public int MonChamp   // propriete publique
{
    get => _monChamp;  // lire la valeur
    set                // ecrire la valeur avec validation
    {
        // Ajouter ici votre validation
        // Si invalide : throw new ArgumentException("message d'erreur");
        _monChamp = value;
    }
```

```
}
```

Les 4 validations obligatoires dans votre projet :

1. Age dans Personne : l'age NE PEUT PAS etre negatif
2. Valeur dans Note : la note DOIT etre entre 0 et 20
3. Salaire dans Formateur: le salaire DOIT etre strictement positif (> 0)
4. Duree dans Module : la duree DOIT etre strictement superieure a 0

Chacune de ces validations doit lever une exception ArgumentException avec un message d'erreur clair si la valeur n'est pas valide.

3.3 Les membres static

Un membre static appartient a la classe elle-meme, pas a une instance. Il est partage par tous les objets de la classe.

Cas d'usage dans votre projet :

Exemple : un compteur statique dans Personne qui s'incremente a chaque creation d'une nouvelle personne (dans le constructeur).

Exemple : un compteur statique dans Module qui s'incremente a chaque creation d'un nouveau module.

Ces compteurs permettent d'afficher les statistiques sans avoir a parcourir des listes.

Syntaxe : private static int _compteur = 0;
Propriete : public static int NombreTotal => _compteur;

4. GUIDE : LIRE ET ECRIRE DES FICHIERS EN C#

Pourquoi sauvegarder dans des fichiers ?

Sans sauvegarde, toutes vos donnees disparaissent a chaque fermeture du programme.
Les fichiers JSON sont recommandes : lisibles, simples et faciles a deboguer.
Format alternatif accepte : fichier texte structure avec separateur (ex: point-virgule).

4.1 Format JSON — comprendre la structure

JSON (JavaScript Object Notation) est un format texte pour stocker des donnees structurees.
Visual Studio peut ouvrir les fichiers JSON pour que vous puissiez les lire et les verifier.

Un fichier etudiants.json ressemble a ceci :

```
[
  {
    "Id": 1,
    "Nom": "Diallo",
    "Prenom": "Fatou",
    "Age": 22,
    "Matricule": "ETU001",
    "Niveau": "L2"
  },
  {
    "Id": 2,
    "Nom": "Sow",
    "Prenom": "Ibrahima",
    "Age": 24,
    "Matricule": "ETU002",
    "Niveau": "L3"
  }
]
```

4.2 Installer Newtonsoft.Json

Newtonsoft.Json est la librairie recommandee pour travailler avec du JSON en C#. Pour l'installer :

1. Dans Visual Studio : Outils > Gestionnaire de packages NuGet > Gerer les packages
2. Rechercher : Newtonsoft.Json
3. Cliquer sur Installer
4. Ajouter au debut du fichier : using Newtonsoft.Json;

4.3 Les operations essentielles sur les fichiers

Ecrire dans un fichier (sauvegarder)

Etapes pour sauvegarder une liste en JSON :

1. Convertir votre liste en texte JSON avec :
   JsonConvert.SerializeObject(maListe, Formatting.Indented)

2. Ecrire ce texte dans un fichier avec :
   File.WriteAllText("nomFichier.json", texteJson)

Conseil : creer une methode dans votre classe Sauvegarde.cs :
public static void SauvegarderEtudiants(List<Etudiant> liste) { ... }

Lire depuis un fichier (charger)

Etapes pour charger une liste depuis un fichier JSON :

1. Verifier que le fichier existe avec :
   File.Exists("nomFichier.json") -> retourne true ou false
   Si le fichier n'existe pas, retourner une liste vide.

2. Lire le contenu texte du fichier avec :
   File.ReadAllText("nomFichier.json")

3. Convertir le texte JSON en liste d'objets avec :
   JsonConvert.DeserializeObject<List<Etudiant>>(texteJson)

Conseil : gerer le cas ou le fichier est vide ou corrompu avec try/catch.

4.4 Quand sauvegarder ?

Moment	Action recommandee
Au demarrage de l'application	Charger toutes les donnees depuis les fichiers
Apres chaque ajout	Sauvegarder immediatement la liste modifiee
Apres chaque modification	Sauvegarder immediatement la liste modifiee
Apres chaque suppression	Sauvegarder immediatement la liste modifiee
A la fermeture (event FormClosing)	Sauvegarder toutes les listes en securite

4.5 Alternative sans NuGet : fichier texte structure

Si vous ne pouvez pas utiliser NuGet, vous pouvez utiliser un fichier texte ou chaque ligne represente un enregistrement, avec les champs separes par un point-virgule.

Principe du fichier texte structure :

Chaque etudiant sur une ligne, champs separes par ';' :
1;Diallo;Fatou;22;ETU001;L2
2;Sow;Ibrahima;24;ETU002;L3

Pour ecrire : utiliser StreamWriter en boucle sur votre liste
Pour lire : utiliser File.ReadAllLines() puis Split(';') sur chaque ligne
Attention a bien parser les types (int.Parse pour les entiers)

Classes utiles : StreamWriter, File.ReadAllLines(), string.Split()

4.6 Structure recommandee de Sauvegarde.cs

Votre classe Sauvegarde doit etre static et contenir :

- Les chemins des fichiers (private static string _cheminEtudiants = ...)
- SauvegarderEtudiants(List<Etudiant> liste)
- ChargerEtudiants() -> retourne List<Etudiant>
- SauvegarderFormateurs(List<Formateur> liste)
- ChargerFormateurs() -> retourne List<Formateur>
- SauvegarderModules(List<Module> liste)
- ChargerModules() -> retourne List<Module>
- SauvegarderNotes(List<Note> liste)
- ChargerNotes(List<Etudiant> etus, List<Module> mods) -> retourne List<Note>

5. GUIDE : INTERFACE GRAPHIQUE WINFORMS

REGLE FONDAMENTALE :

Separez bien la logique metier de l'interface graphique !

Les classes metier (Etudiant, Formateur, GestionCentre...) ne doivent JAMAIS contenir de code WinForms (MessageBox, TextBox, Button...).

Les formulaires ne doivent JAMAIS contenir de logique de calcul (pas de calcul de moyenne dans un formulaire, cela va dans Etudiant).

Les formulaires font uniquement : recuperer les saisies, appeler les methodes metier, et afficher les resultats.

5.1 Structure des formulaires recommandee

Formulaire	Role	Elements cles
FormMain.cs	Fenetre principale	TabControl ou menu principal
FormAjoutEtudiant.cs	Ajouter / modifier	TextBox, ComboBox, bouton Valider
FormAjoutFormateur.cs	Ajouter un formateur	TextBox, NumericUpDown, bouton Valider
FormAjoutModule.cs	Ajouter un module	TextBox, NumericUpDown, bouton Valider
FormNotes.cs	Gerer les notes	ComboBox etudiant, ComboBox module, NumericUpDown
FormStatistiques.cs	Voir les stats	Labels, DataGridView ou ListBox

5.2 Controles WinForms utiles

Controle	Usage	Exemple d'utilisation
TextBox	Saisie de texte libre	Nom, Prenom, Matricule, Specialite
Label	Affichage de texte fixe	Etiquettes de champs, resultats calculcs
Button	Declencher une action	Ajouter, Modifier, Supprimer, Fermer
ComboBox	Liste deroulante (1 seul choix)	Niveau (L1/L2/L3/M1/M2), Module, Etudiant
NumericUpDown	Saisie numerique avec fleches	Age, Note (entre 0 et 20), Salaire
DataGridView	Tableau de donnees	Listes d'etudiants, notes, modules
ListBox	Liste simple	Affichage des resultats, selection
TabControl	Navigation par onglets	Un onglet par section : Etudiants, Formateurs...
GroupBox	Regrouper des controles	Grouper les champs d'un formulaire

5.3 Fonctionnement des dialogues (ShowDialog)

Lorsqu'un formulaire secondaire (ex: FormAjoutEtudiant) est ouvert depuis le formulaire principal, on utilise ShowDialog() pour l'ouvrir en mode modal.

Principe du dialogue entre formulaires :

1. Dans FormMain : ouvrir le formulaire secondaire avec ShowDialog()
   Si le resultat est DialogResult.OK : recuperer les donnees saisies

2. Dans FormAjoutEtudiant : quand l'utilisateur clique sur 'Valider' :
   - Recuperer les valeurs des TextBox, ComboBox, etc.
   - Creer l'objet Etudiant avec ces valeurs
   - Definir DialogResult = DialogResult.OK
   - Fermer le formulaire avec Close()

3. Si une validation echoue (age negatif, champ vide...) :
   - Afficher un MessageBox avec le message d'erreur
   - NE PAS fermer le formulaire (laisser l'utilisateur corriger)

4. Pour passer des donnees entre formulaires :
   - Creer une propriete publique dans le formulaire secondaire
   - Le formulaire principal lit cette propriete apres ShowDialog()

5.4 Affichage avec DataGridView

Pour afficher une liste dans un DataGridView :

1. Vider la source de donnees avant de la reassigner :
   dataGridView.DataSource = null;
   (sans ce null, l'affichage peut ne pas se rafraichir)

2. Assigner la nouvelle source :
   dataGridView.DataSource = maListe;

3. Pour afficher uniquement certaines colonnes (recommande) :
   Utiliser LINQ avec .Select() pour projeter les champs voulus
   Ex: afficher Id, Nom, Prenom, Niveau, Moyenne calculee

4. Appeler ActualiserAffichage() apres chaque modification de la liste.

5.5 Gestion des erreurs dans les formulaires

Toujours entourer la creation d'objets avec try/catch :

Pourquoi ? Parce que vos validations dans les set lancent des exceptions.

Si vous ne les attrapez pas, le programme plantera.

Principe :

```
try {
    // Creer l'objet avec les valeurs saisies
    // Si une validation echoue, une exception est levee automatiquement
}
catch (Exception ex) {
    // Afficher le message d'erreur a l'utilisateur
    MessageBox.Show(ex.Message, "Erreur de saisie", ...);
    // Ne pas fermer le formulaire
}
```

6. FONCTIONNALITES A IMPLEMENTER

Voici la liste complete des fonctionnalites que votre application doit proposer. Implementez-les une par une en testant chacune avant de passer a la suivante.

6.1 Gestion des Etudiants

- Ajouter un etudiant : formulaire avec tous les champs + validation
- Modifier un etudiant : selectionner dans le tableau, pre-remplir le formulaire
- Supprimer un etudiant : avec MessageBox de confirmation (Oui / Non)
- Afficher la liste : DataGridView avec colonnes Id, Nom, Prenom, Niveau, Moyenne
- (Bonus) Recherche par nom ou par niveau

6.2 Gestion des Formateurs

- Ajouter un formateur : nom, prenom, specialite, salaire, module enseigne
- Afficher la liste des formateurs
- (Bonus) Modifier / supprimer un formateur

6.3 Gestion des Modules

- Ajouter un module : code, libelle, duree (heures), coefficient
- Afficher la liste des modules
- (Bonus) Modifier / supprimer un module

6.4 Gestion des Notes

- Selectionner un etudiant et un module via des ComboBox
- Saisir une note avec NumericUpDown (min 0, max 20)
- Afficher toutes les notes d'un etudiant selectionne
- Calculer et afficher la moyenne de cet etudiant en temps reel
- (Bonus) Modifier ou supprimer une note existante

6.5 Statistiques

- Nombre total d'etudiants (via membre static)
- Nombre total de formateurs
- Nombre total de modules (via membre static)
- Moyenne generale de tous les etudiants
- Meilleur etudiant : afficher son nom et sa moyenne
- (Bonus) Classement des etudiants par moyenne decroissante
- (Bonus) Etudiant avec la note la plus basse

7. ORGANISATION DU PROJET

L'organisation de votre code est un critere evalue. Un projet bien organise est plus facile a deboguer, a presenter et a maintenir.

7.1 Structure recommandee

Organisation des fichiers de votre solution Visual Studio :

```
CentreFormation/  (dossier racine du projet)
|
+-- Models/                (classes metier)
|   +-- Personne.cs
|   +-- Etudiant.cs
|   +-- Formateur.cs
|   +-- Module.cs
|   +-- Note.cs
|
+-- Services/             (logique applicative)
|   +-- GestionCentre.cs    <- listes + methodes metier
|   +-- Sauvegarde.cs       <- lecture / ecriture fichiers
|
+-- Forms/                (interface graphique)
|   +-- FormMain.cs
|   +-- FormAjoutEtudiant.cs
|   +-- FormAjoutFormateur.cs
|   +-- FormAjoutModule.cs
|   +-- FormNotes.cs
|   +-- FormStatistiques.cs
|
+-- Data/                 (fichiers de sauvegarde JSON)
|   +-- etudiants.json
|   +-- formateurs.json
|   +-- modules.json
|   +-- notes.json
|
+-- Program.cs              <- point d'entree MINIMAL
+-- README.md
```

7.2 Programme.cs — point d'entree minimal

Program.cs ne doit contenir que le strict minimum :

- La methode Main() statique
- L'appel a Application.EnableVisualStyles()
- L'appel a Application.Run(new FormMain())

AUCUNE logique metier dans Program.cs !
Si vous avez plus de 5 lignes dans Program.cs, quelque chose ne va pas.

7.3 Separation des responsabilites

Fichier	Ce qu'il contient	Ce qu'il ne doit PAS contenir
Etudiant.cs	Attributs, proprietes, CalculerMoyenne()	Tout code WinForms
Formateur.cs	Attributs, proprietes, CalculerSalaireAnnuel()	Tout code WinForms
GestionCentre.cs	Listes, methodes metier, MeilleurEtudiant()	Tout code WinForms
Sauvegarde.cs	Lire/ecrire fichiers uniquement	Logique metier
FormMain.cs	Navigation, appels aux methodes metier	Calculs, validations metier
FormAjoutEtudiant.cs	Recuperer saisie, creer objet Etudiant	Logique de calcul

8. CHECKLIST AVANT LA REMISE

Verifiez chaque point avant d'envoyer votre projet !

Cochez mentalement chaque element. Si un point est manquant, completez-le.

Programmation Orientee Objet

- [ ] Classe Personne avec Id, Nom, Prenom, Age, constructeur, AfficherDetails()
- [ ] Classe Etudiant heritee de Personne (mot-cle : base(...))
- [ ] Classe Formateur heritee de Personne (mot-cle : base(...))
- [ ] Classe Module avec toutes les proprietes
- [ ] Classe Note avec reference a Etudiant et Module
- [ ] Au moins un membre static pertinent (compteur de personnes ou modules)
- [ ] Methode CalculerMoyenne() dans Etudiant
- [ ] Methode CalculerSalaireAnnuel() dans Formateur
- [ ] Override de AfficherDetails() dans Etudiant et Formateur

Encapsulation — Validations dans les set

- [ ] Age >= 0 dans Personne
- [ ] Valeur entre 0 et 20 dans Note
- [ ] Salaire > 0 dans Formateur
- [ ] Duree > 0 dans Module
- [ ] Chaque validation leve une ArgumentException avec un message clair

Interface WinForms

- [ ] Formulaire principal avec navigation claire (onglets ou menu)
- [ ] Ajouter un etudiant (formulaire avec validation)
- [ ] Modifier un etudiant
- [ ] Supprimer un etudiant (avec confirmation)
- [ ] Ajouter un formateur
- [ ] Ajouter un module
- [ ] Attribuer une note a un etudiant
- [ ] Afficher la moyenne d'un etudiant
- [ ] Afficher les statistiques
- [ ] Gestion des erreurs avec try/catch + MessageBox dans les formulaires

Sauvegarde

- [ ] Chargement des donnees au demarrage
- [ ] Sauvegarde apres chaque ajout, modification, suppression
- [ ] Fichiers JSON generes et presents dans le dossier Data/

- [ ] Verification de l'existence du fichier avant lecture (File.Exists)

Organisation du code

- [ ] Chaque classe dans son propre fichier .cs
- [ ] Separation logique metier / formulaires respectee
- [ ] Program.cs minimal
- [ ] README.md inclus avec instructions de lancement

9. INSTRUCTIONS DE REMISE

Envoi par email

Adresse : aboubacarsawadogo15@gmail.com

Objet EXACT : Projet dotnet c# 2026

(Tout email avec un autre objet ne sera pas pris en compte)

Format : un seul fichier .zip ou .rar contenant tout le dossier Visual Studio

Comment preparer votre archive

5. Fermez Visual Studio completement
6. Ouvrez votre dossier de projet (ex: Documents > Visual Studio > CentreFormation)
7. Supprimez les dossiers bin\ et obj\ pour reduire la taille (ils se regenerent)
8. Clic droit sur le dossier > Envoyer vers > Dossier compresse (zippe)
9. Renommez l'archive : VotreNom_ProjetDotnet.zip
10. Envoyez l'archive par email avec l'objet EXACT : Projet dotnet c# 2026

Contenu du README.md a rediger

Votre README.md doit repondre a ces questions :

1. Qui etes-vous ? (Prenom, NOM, classe)
2. Comment lancer le projet ? (fichier .sln a ouvrir, touche pour demarrer)
3. Quelles sont les fonctionnalites implementees ?
4. Y a-t-il des dependances ? (ex: Newtonsoft.Json via NuGet)
5. Y a-t-il des remarques particulieres sur votre projet ?

10. CONSEILS ET ERREURS COURANTES

10.1 Ordre de developpement recommande

11. Creez d'abord toutes les classes metier (Models/) — sans interface
12. Testez votre logique avec quelques lignes dans Program.cs (console)
13. Une fois la logique validee, effacez le code de test
14. Ajoutez la classe Sauvegarde.cs et testez lecture/ecriture
15. Creez le FormMain avec sa navigation (onglets ou menu)
16. Creez chaque formulaire secondaire un par un
17. Apres chaque formulaire : testez-le avant de passer au suivant
18. Ajoutez les statistiques en dernier

10.2 Erreurs frequentes a eviter

Erreurs courantes qui coutent des points :

- Mettre la logique metier dans les formulaires
  -> Deplacer les calculs dans les classes (Etudiant, GestionCentre...)

- Oublier de sauvegarder apres une modification
  -> Appeler la methode de sauvegarde juste apres chaque modification

- Ne pas gerer les exceptions avec try/catch lors des saisies
  -> Toute creation d'objet doit etre entouree d'un try/catch

- Utiliser des champs publics au lieu de proprietes avec get/set
  -> Toujours utiliser des proprietes, jamais de champs publics directs

- Oublier d'appeler base(...) dans les constructeurs des classes filles
  -> Etudiant et Formateur doivent toujours appeler : base(id, nom, prenom, age)

- Ne pas vider le DataSource avant de le reassigner
  -> Faire : dgv.DataSource = null; avant dgv.DataSource = maListe;

- Ne pas verifier si le fichier existe avant de le lire
  -> Toujours faire File.Exists(...) avant File.ReadAllText(...)

Bonnes pratiques qui font la difference :

+ Validez toujours les saisies avec try/catch ET MessageBox
+ Utilisez des MessageBox de confirmation avant de supprimer
+ Commentez votre code avec // pour expliquer ce que vous faites
+ Utilisez des noms de variables clairs (pas a, b, x ou temp1)
+ Testez chaque fonctionnalite avant de passer a la suivante
+ Relancez l'application pour verifier que la sauvegarde fonctionne

+ Verifiez que les fichiers JSON sont bien crees dans votre dossier

10.3 Conseils pour la presentation orale

- Relancez votre application au moins une fois la veille de la soutenance
- Preparez un jeu de donnees de test : quelques etudiants, formateurs, notes
- Sachez expliquer chaque classe et ce qu'elle represente
- Sachez montrer : l'heritage, une validation, un membre static, une methode metier
- Anticipez les questions : 'Pourquoi un static ici ?', 'Que fait CalculerMoyenne ?'
- Si une fonctionnalite ne marche pas, soyez honnete et expliquez le probleme

Bonne chance a tous !

Un projet incomplet, sans interface, ou que vous ne pouvez pas expliquer sera penalise.